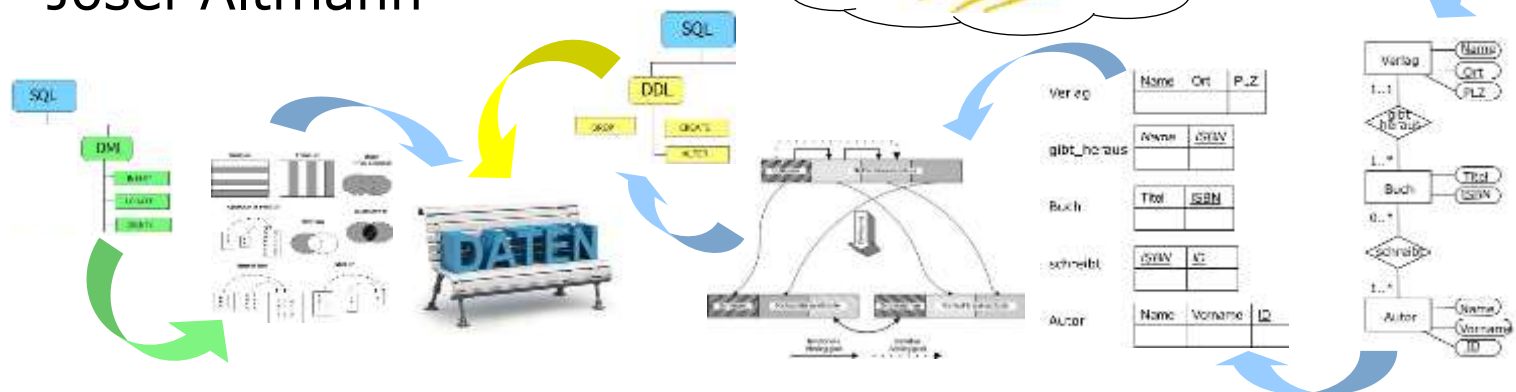




Klausurvorbereitung

Josef Altmann





Präambel/Hinweis

- Die Beispiele dienen zur Wiederholung ausgewählter Inhalte und stellen eine Vorbereitung für die Vorlesungsklausur dar.
- Beachten Sie, dass dieser Foliensatz nicht alle für die Vorlesungsklausur relevanten Inhalte abdeckt.

Entity Relationship Diagramm

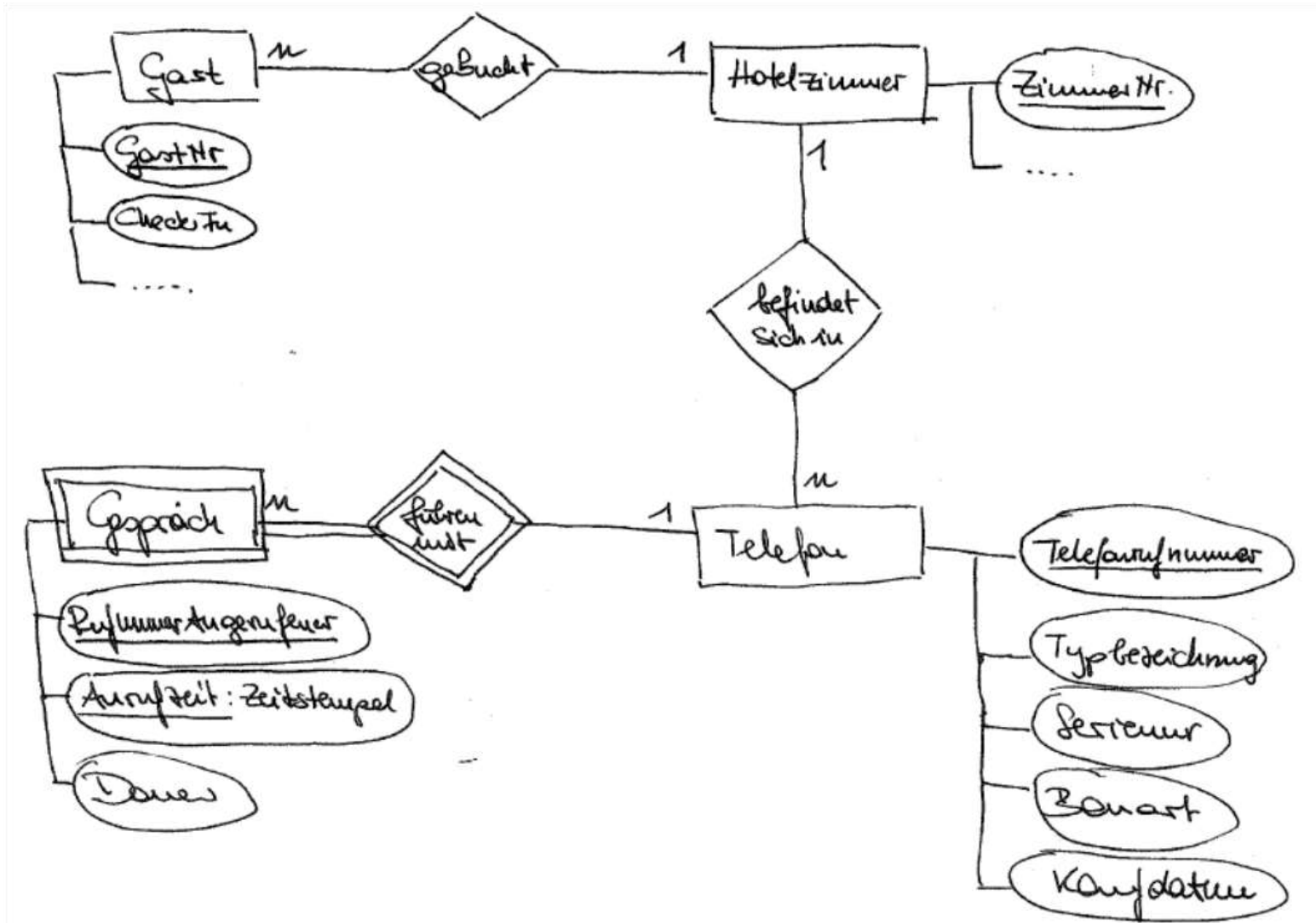
Telefonabrechnung

■ Modellierung eines ER-Modells

- Die Abrechnung einer internen Telefonanlage eines Hotels soll auf eine Datenbanklösung umgestellt werden. Jedes Hotelzimmer verfügt über mindestens ein Telefon und einen Gast, der in dieser Zeit auf dieses Zimmer gebucht ist. Über Telefone ist die Seriennummer, die Typenbezeichnung, die Bauart und das Kaufdatum zu speichern. Alle Gespräche sollen so aufgezeichnet werden, dass eine eindeutige Abrechnung der Gebühren gegenüber den Hotelgästen erfolgen kann. Hotelinterne Gespräche sind kostenlos.
- Geben Sie im ER-Diagramm alle Attribute (inkl. PK) und alle Kardinalitäten an und beschriften Sie die Beziehungen. Dokumentieren Sie alle Annahmen, die Sie treffen.

Entity Relationship Diagramm

Telefonabrechnung



Entity Relationship Diagramm

Bibliotheksverwaltung

■ Modellierung eines ER-Modells

- Eine Bibliothek besteht aus Büchern und Zeitschriften. Jedes Buch kann ggf. mehrere Autoren haben und ist eindeutig durch seine ISBN gekennzeichnet. Die Bibliothek besitzt teilweise mehrere Exemplare eines Buches. Zeitschriften dagegen sind jeweils nur einmal vorhanden. Sie erscheinen in einzelnen Heften und werden jahrgangsweise gebunden. Die in Zeitschriften publizierten Artikel sind ebenso wie Bücher einem oder mehreren Fachgebieten (z.B. Programmiersprachen, Kommunikationswissenschaften) zugeordnet. Ausgeliehen werden können nur Bücher (keine Zeitschriften).
- Geben Sie im ER-Diagramm alle Attribute (inkl. PK) und alle Kardinalitäten an und beschriften Sie die Beziehungen. Dokumentieren Sie alle Annahmen, die Sie treffen.

Entity Relationship Diagramm

Bibliotheksverwaltung

■ Modellierung eines ER-Modells

- Eine Bibliothek besteht aus **Büchern** und **Zeitschriften**. Jedes **Buch** kann ggf. mehrere **Autoren** haben und ist eindeutig durch seine ISBN gekennzeichnet. Die Bibliothek besitzt teilweise mehrere **Exemplare** eines **Buches**. **Zeitschriften** dagegen sind jeweils nur einmal vorhanden. Sie erscheinen in einzelnen **Heften** und werden jahrgangsweise gebunden. Die in **Zeitschriften** publizierten **Artikel** sind ebenso wie **Bücher** einem oder mehreren **Fachgebieten** (z.B. Programmiersprachen, Kommunikationswissenschaften) zugeordnet. Ausgeliehen werden können nur Bücher (keine Zeitschriften).
- Geben Sie im ER-Diagramm alle Attribute (inkl. PK) und alle Kardinalitäten an und beschriften Sie die Beziehungen. Dokumentieren Sie alle Annahmen, die Sie treffen.

Entity Relationship Diagramm

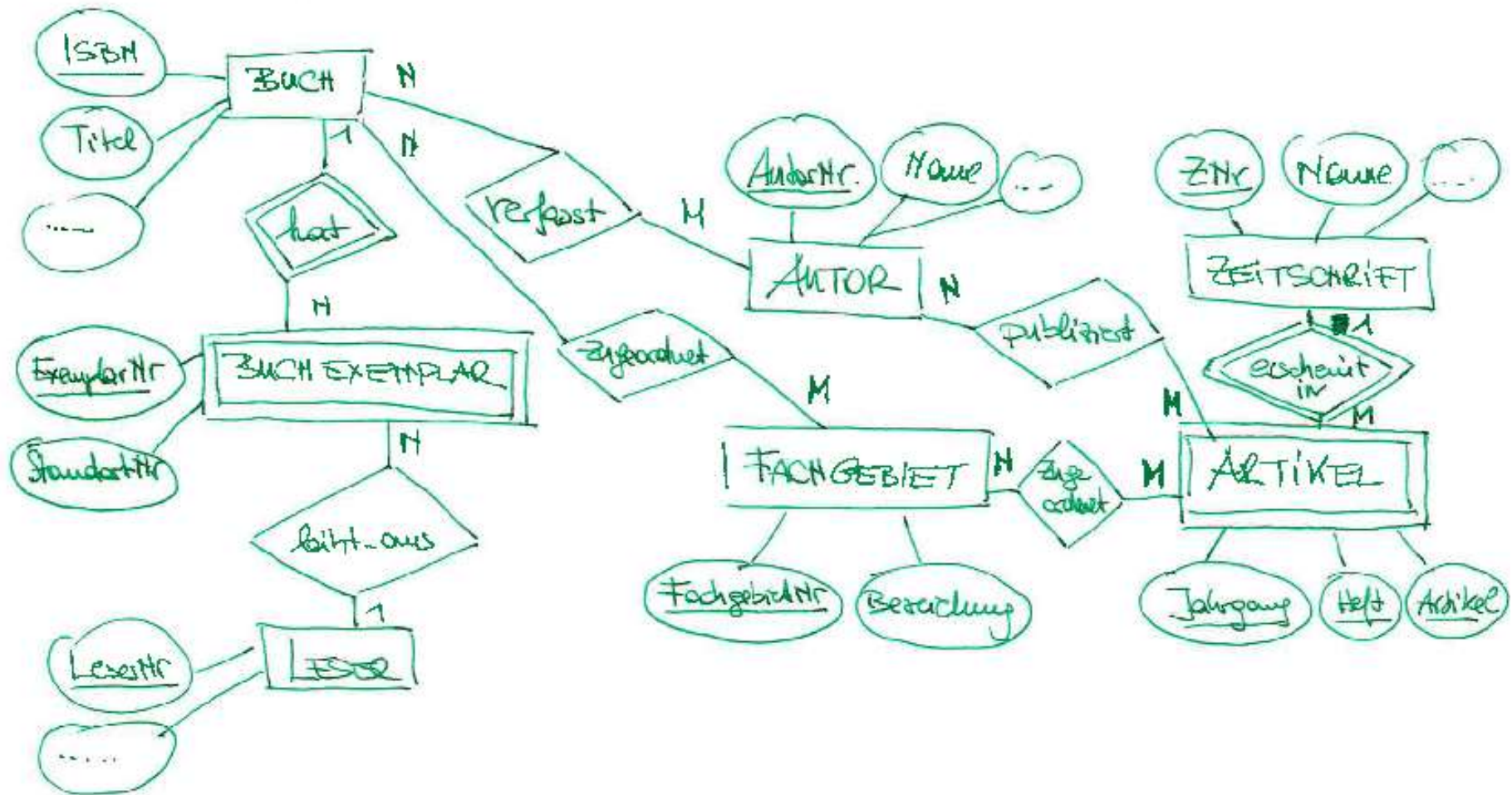
Bibliotheksverwaltung

■ Modellierung eines ER-Modells

- Eine Bibliothek besteht aus **Büchern** und **Zeitschriften**. Jedes **Buch** kann ggf. mehrere **Autoren** haben und ist eindeutig durch seine **ISBN** gekennzeichnet. Die Bibliothek besitzt teilweise mehrere **Exemplare** eines **Buches**. **Zeitschriften** dagegen sind jeweils nur einmal vorhanden. Sie erscheinen in einzelnen **Heften** und werden jahrgangsweise gebunden. Die in **Zeitschriften** publizierten **Artikel** sind ebenso wie **Bücher** einem oder mehreren **Fachgebieten** (z.B. **Programmiersprachen**, **Kommunikationswissenschaften**) zugeordnet. Ausgeliehen werden können nur Bücher (keine Zeitschriften).
- Geben Sie im ER-Diagramm alle Attribute (inkl. PK) und alle Kardinalitäten an und beschriften Sie die Beziehungen. Dokumentieren Sie alle Annahmen, die Sie treffen.

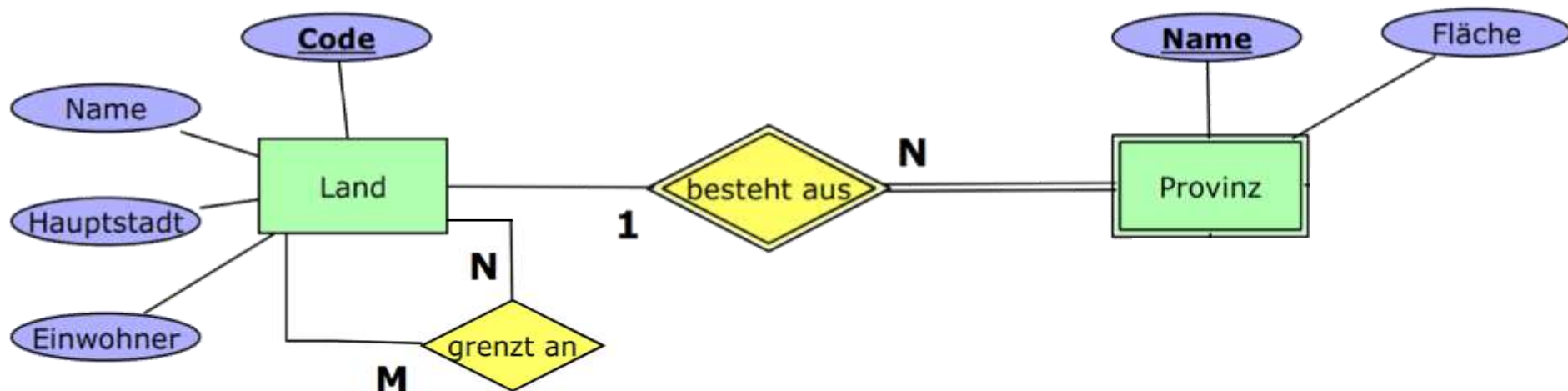
Entity Relationship Diagramm

Bibliotheksverwaltung

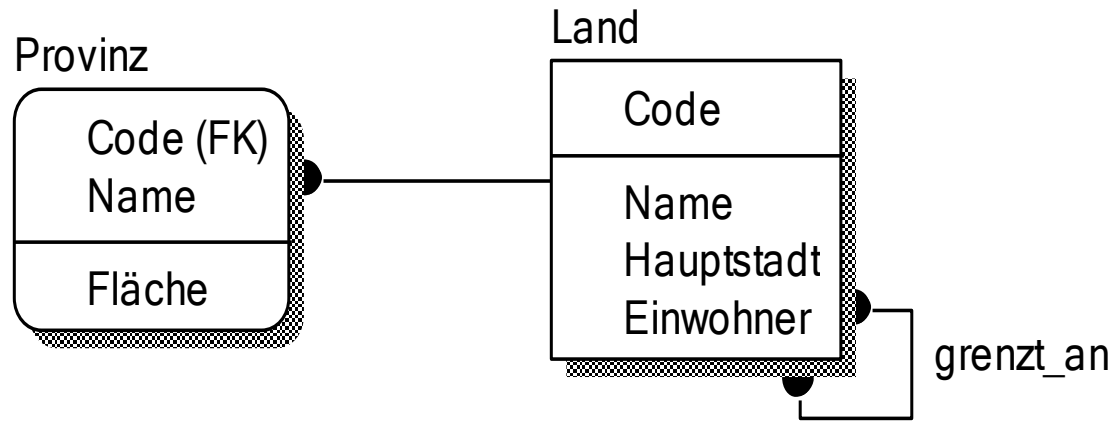


Logisches Modell

- Transformieren Sie das angegebene ER-Modell in ein entsprechendes logisches (= relationales) Modell, das der 3. Normalform entspricht. Kennzeichnen Sie im logischen Schemata die jeweiligen Primär- und Fremdschlüsseln bzw. Primär-/Fremdschlüsseln. Die Darstellung des logisches Modells hat in der folgenden Notation zu erfolgen: Relationenname (Attribut1:Datentyp1, ...) – wählen Sie dafür passende Datentypen.



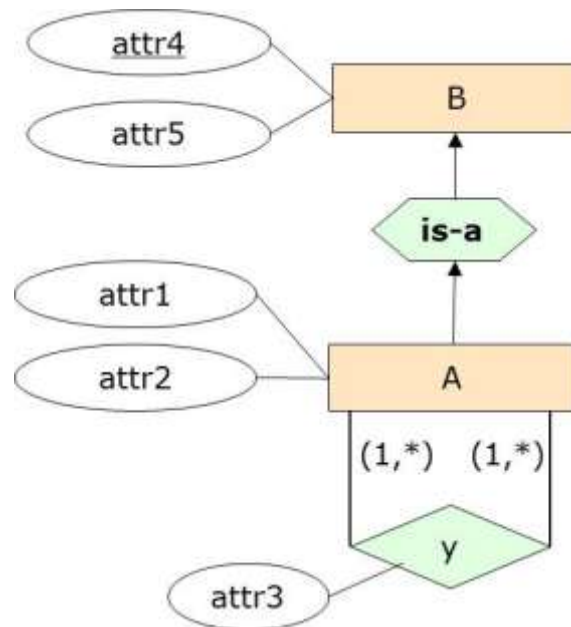
Logisches Modell



- **Land** (Code:int, Name:string, Hauptstadt:string, Einwohner:int)
- **Provinz** (Code:int, Name:string, Fläche:int)
- **grenzt_an** (Land.Code1:int, Land.Code2:int)

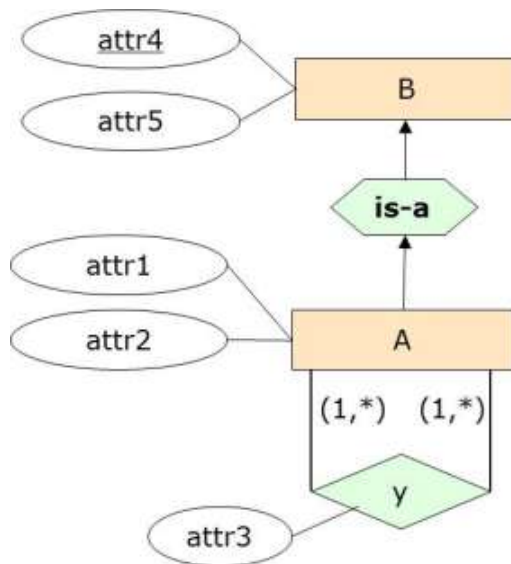
Logisches Modell

- Führen Sie das folgende ER-Diagramm in ein Relationenmodell über – vergessen Sie nicht, auch die Primär- und Fremd-schlüssel zu kennzeichnen. Verwenden Sie möglichst wenig Relationen und beachten Sie, dass Nullwerte vermieden werden sollten.



Logisches Modell

- Führen Sie das folgende ER-Diagramm in ein Relationenmodell über – vergessen Sie nicht, auch die Primär- und Fremd-schlüssel zu kennzeichnen. Verwenden Sie möglichst wenig Relationen und beachten Sie, dass Nullwerte vermieden werden sollten.



Lösung

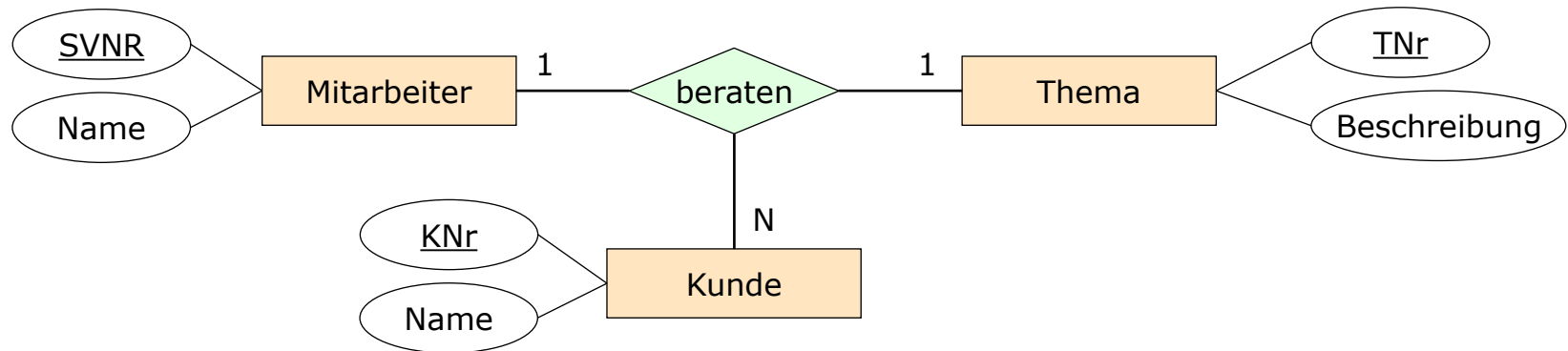
```
B ( attr4, attr5)
```

```
A ( B.attr4, attr1, attr2 )
```

```
y ( A.A1attr4, A.A2attr4, attr3)
```

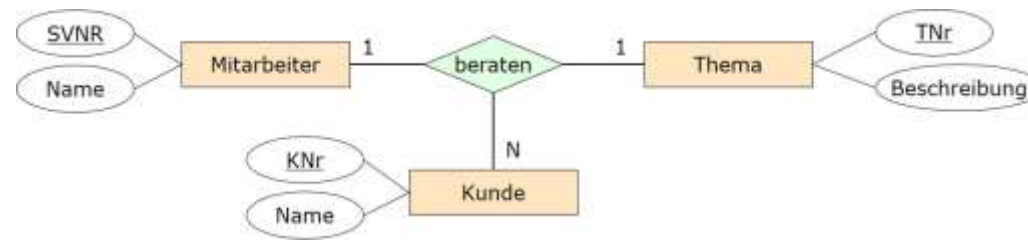
Logisches Modell

- Gegeben ist folgender Ausschnitt eines ER-Diagramms mit Kardinalitätsangaben.



- Beschreiben Sie die daraus abzuleitenden Integritätsbedingungen in eigenen Worten.

Logisches Modell



- Beschreiben Sie die daraus abzuleitenden Integritätsbedingungen in eigenen Worten.
 - beraten: Mitarbeiter x Kunde → Thema
in Worten: ein Mitarbeiter berät einen Kunden zu genau einem Thema.
 - beraten: Thema x Kunde → Mitarbeiter
in Worten: ein Kunde wird zu einem Thema von genau einem Mitarbeiter beraten.
 - Ein Mitarbeiter berät zu einem Thema mehrere Kunden

- Ergänzen Sie das Relationenmodell
 - Mitarbeiter (.....)
 - Kunde (.....)
 - Thema (.....)
 - beraten (.....)
 - Mitarbeiter (SVNR, Name)
 - Kunde (KNr, Name)
 - Thema (TNr, Beschreibung)
 - beraten (KNr, SVNR, TNr)
 - [beraten (KNr, TNr, SVNR)]

Funktionale Abhängigkeiten

- Funktionale Abhängigkeit zwischen Attributmengen α und β einer Relation gilt,
 - wenn in jedem Tupel der Relation der Attributwert unter den α -Komponenten den Attributwert unter den β -Komponenten festlegt.
- „Die α -Werte bestimmen die β -Werte eindeutig (funktional).“
- „ β -Werte sind funktional abhängig von den α -Werten.“

Funktionale Abhängigkeit:

$\alpha \rightarrow \beta$ genau dann wenn $\forall r, s \in R$ mit $r.\alpha = s.\alpha \Rightarrow r.\beta = s.\beta$

Funktionale Abhängigkeiten

- Eine funktionale Abhängigkeit $\alpha \rightarrow \beta$ ist dann trivial, wenn β eine Teilmenge von α ist.
 - wahr ☐
 - falsch ☐
 - weiß nicht ☐

- Eine funktionale Abhängigkeit $\alpha \rightarrow \beta$ ist dann trivial, wenn β eine Teilmenge von ist.
 - wahr ☒
 - falsch ☐
 - weiß nicht ☐

Funktionale Abhängigkeiten

- Betrachten Sie den Datenbestand von $R = ABCD$ in folgender Tabelle.

A	B	C	D
7	3	8	4
9	4	3	2
9	2	1	4
7	9	8	3

- Welche der folgenden funktionalen Abhängigkeiten gelten in R ?

	gilt	gilt nicht	weiß nicht
● $AB \rightarrow C$	☐	☐	☐
● $B \rightarrow C$	☐	☐	☐
● $A \rightarrow C$	☐	☐	☐
● $D \rightarrow A$	☐	☐	☐

Funktionale Abhängigkeiten

- Betrachten Sie den Datenbestand von $R = ABCD$ in folgender Tabelle.

A	B	C	D
7	3	8	4
9	4	3	2
9	2	1	4
7	9	8	3

- Welche der folgenden funktionalen Abhängigkeiten gelten in R ?

	gilt	gilt nicht	weiß nicht
● $AB \rightarrow C$	☒	☐	☐
● $B \rightarrow C$	☒	☐	☐
● $A \rightarrow C$	☐	☒	☐
● $D \rightarrow A$	☐	☒	☐

Schlüssel

- Ein Relationenschema R kann zwei verschiedene Schlüssel K_1 und K_2 besitzen, für die gilt: $|K_1| < |K_2|$
 - wahr ☐
 - falsch ☐
 - weiß nicht ☐

Schlüssel

- Ein Relationenschema R kann zwei verschiedene Schlüssel K_1 und K_2 besitzen, für die gilt: $|K_1| < |K_2|$

- wahr ☒
- falsch ☐
- weiß nicht ☐

Normalformen

- Überführen Sie nachfolgende Relation **Professor_lehrt_in_Studiengang** in die dritte Normalform.
- Geben Sie die Relationenschemata an, die sich aus dem Normalisierungsprozess ergeben und kennzeichnen Sie die jeweiligen Primär- und Fremdschlüsseln bzw. Primär-/Fremdschlüsseln.
- Annahme: Ein Professor kann in mehreren Studiengängen lehren. **PLZ** (Postleitzahl) und **ort** geben den Hauptwohnsitz des Professors an – ein Professor kann nur einen Hauptwohnsitz besitzen

Professor_lehrt_in_Studiengang

<u>Studiengangskennzahl</u>	<u>Personalnr.</u>	Vorname	Nachname	PLZ	Ort	Studiengangsbezeichnung
0456	P20621	Josef	Altmann	4040	Linz	Kommunikation, Wissen, Medien (BA)
0454	P20888	Erik	Pitzer	4600	Wels	Software Engineering (BA)
0454	P20621	Josef	Altmann	4040	Linz	Software Engineering (BA)
0307	P20621	Josef	Altmann	4040	Linz	Software Engineering (MA)

Normalformen

Professor lehrt in Studiengang

<u>Studiengangskennzahl</u>	<u>Personalnr.</u>	Vorname	Nachname	PLZ	Ort	Studiengangsbezeichnung
0456	P20621	Josef	Altmann	4040	Linz	Kommunikation, Wissen, Medien (BA)
0454	P20888	Erik	Pitzer	4600	Wels	Software Engineering (BA)
0454	P20621	Josef	Altmann	4040	Linz	Software Engineering (BA)
0307	P20621	Josef	Altmann	4040	Linz	Software Engineering (MA)

- Annahme: Ein Professor kann in mehreren Studiengängen lehren. PLZ (Postleitzahl) und Ort geben den Hauptwohnsitz des Professors an – ein Professor kann nur einen Hauptwohnsitz besitzen
- Professor (Personalnr, Vorname, Nachname PLZ)
- Ort (PLZ, Ort)
- Studiengang(Studiengangskennzahl, Studiengangsbezeichnung)
- Studiengang_Professor (Studiengangskennzahl, Personalnr)

Normalformen

LEGENDE:

• BNr	<i>BuchungsNummer integer</i>	• von	<i>Buchung ab Datum date</i>
• NName	<i>Nachname Gast varchar(100)</i>	• bis	<i>Buchung bis Datum date</i>
• VName	<i>Vorname Gast varchar(50)</i>	• Typ	<i>Zimmertyp integer</i>
• Mail	<i>E-Mail Gast varchar(30)</i>	• BDatum	<i>Datum der Buchung date</i>
• Tel	<i>Telefonnummer Gast varchar (20)</i>	• Bad	<i>Ausstattung mit Bad char(1) ['J'/'N']</i>
• Raum	<i>Zimmernummer integer</i>	• WC	<i>Ausstattung mit WC char(1) ['J'/'N']</i>
		• Betten	<i>Anzahl Betten im Zimmer integer</i>

- Die nachfolgende Relation **HOTEL** befindet sich in 1NF. Sie besitzt nur einen künstlichen Primärschlüssel (BNr), um Buchungen eindeutig zu identifizieren. Inhaltlich beschreibt die Relation **HOTEL** einen Ausschnitt aus einem Hotelbuchungssystem, in dem Gäste Zimmer buchen können. Um die Buchungen datenbank-unterstützt durchführen zu können, soll die Relation in die 3NF überführt werden.

HOTEL(BNr, NName, VName, Mail, Tel, Typ, Raum, von, bis, BDatum, Bad, Betten, WC)

- Entwickeln Sie im Normalisierungsprozess ggf. sinnvolle Primär- und Fremdschlüssel, um die semantischen Beziehungen zwischen den normalisierten Relationen darzustellen. Beachten Sie, dass ein Zimmer immer nur einem Zimmertyp angehören kann und dass die Ausstattung (Bad, WC, Betten) für einen Zimmertyp immer gleich ist. Ein Gast kann selbstverständlich mehrere Zimmer buchen. Es werden aber immer nur eine E-Mail-Adresse und eine Telefonnummer erfasst.

Normalformen

- Gast (GastNr, NName, VName, Mail, Tel)
- Buchung (BNr, BDatum, GastNr, RaumNr, von, bis)
- Zimmer (RaumNr, TypNr)
- Zimmertyp (TypNr, Bad, WC, Betten)

Anmerkung: Obige Lösung ist akzeptabel, denn es war nicht gefordert, dass ein Gast mit einer Buchung mehrere Zimmer bestellen kann. Soll diese Anforderung berücksichtigt werden, dann ist folgende Lösung zu wählen:

- Gast (GastNr, NName, VName, Mail, Tel)
- Buchung (BNr, BDatum, GastNr)
- Buchung_Zimmer (BNr, RaumNr, von, bis)
- Zimmer (RaumNr, TypNr)
- Zimmertyp (TypNr, Bad, WC, Betten)

Normalformen

- Eine Relation $\mathbf{R}$ ist bekanntlich in 3. Normalform, wenn sie keine transitiven Abhängigkeiten eines nicht-primen Attributes von einem Schlüssel hat.
- Für *Schriftklassen* und den enthaltenen *Schriftfamilien* haben wir eine Relation $\mathbf{SK}$ entworfen.
- Ist $\mathbf{SK}$ in 3.NF? Begründung! Bestimmen Sie zuerst die Schlüsselkandidaten und die funktionalen Abhängigkeiten.

SK

Schriftname	KlassenID	Klassenname
Times Roman	RA	Renaissance Antiqua
Garamond	RA	Renaissance Antiqua
Bodoni	KA	Klassizistische Antiqua
MS Arial	LG	Linear Grotesk
Helvetica	LG	Linear Grotesk
Univers	LG	Linear Grotesk

Normalformen

SK

Schriftname	KlassenID	Klassenname
Times Roman	RA	Renaissance Antiqua
Garamond	RA	Renaissance Antiqua
Bodoni	KA	Klassizistische Antiqua
MS Arial	LG	Linear Grotesk
Helvetica	LG	Linear Grotesk
Univers	LG	Linear Grotesk

- Schlüsselkandidaten: {Schriftname}
- Funktionale Abhängigkeiten:
 - {Schriftname} $\rightarrow$ {KlassenID, Klassenname}
 - {KlassenID} $\rightarrow$ {Klassenname}
 - {Klassenname} $\rightarrow$ {KlassenID}
- SK ist nicht in 3.NF, da {KlassenID} $\rightarrow$ {Klassenname} und {KlassenID} kein Schlüssel und Klassenname nicht prim ist.

Fremdschlüssel

Welche der folgenden Aussagen sind wahr, welche sind falsch?

1. Ein Fremdschlüssel legt fest, dass in einer bestimmten Spalte der einen Tabelle Werte verwendet werden können, die als Primärschlüssel der anderen Tabelle vorhanden sind.
2. Die Tabelle, die den Primärschlüssel enthält, wird als Primärtabelle (Elterntabelle) bezeichnet, die Tabelle mit dem Fremdschlüssel als Detailtabelle (Kindtabelle).
3. Ein INSERT in der Detailtabelle ist immer möglich, ohne die Werte in der Primärtabelle zu beachten.
4. Ein UPDATE in der Primärtabelle ist immer möglich, ohne die Werte in der Detailtabelle zu beachten.
5. Ein DELETE in der Detailtabelle ist immer möglich, ohne die Werte in der Primärtabelle zu beachten.
6. Ein DELETE in der Primärtabelle ist immer möglich, ohne die Werte in der Detailtabelle zu beachten.
7. Ein FOREIGN KEY wird so definiert: Er wird der Tabelle mit dem Fremdschlüssel und der betreffenden Spalte zugeordnet und verweist auf die Tabelle mit dem Primärschlüssel und der betreffenden Spalte.
8. Ein FOREIGN KEY kann nicht unmittelbar bei der Definition der Fremdschlüssel-Spalte angegeben werden.
9. Eine Fremdschlüssel-Beziehung kann nur zu jeweils einer Spalte der beiden Tabellen gehören.

Fremdschlüssel

Welche der folgenden Aussagen sind wahr, welche sind falsch?

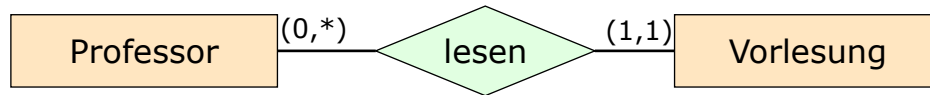
- ✓ Ein Fremdschlüssel legt fest, dass in einer bestimmten Spalte der einen Tabelle Werte verwendet werden können, die als Primärschlüssel der anderen Tabelle vorhanden sind.
- ✓ Die Tabelle, die den Primärschlüssel enthält, wird als Primärtabelle (Elterntabelle) bezeichnet, die Tabelle mit dem Fremdschlüssel als Detailtabelle (Kindtabelle).
- ✗ Ein INSERT in der Detailtabelle ist immer möglich, ohne die Werte in der Primärtabelle zu beachten.
- ✗ Ein UPDATE in der Primärtabelle ist immer möglich, ohne die Werte in der Detailtabelle zu beachten.
- ✓ Ein DELETE in der Detailtabelle ist immer möglich, ohne die Werte in der Primärtabelle zu beachten.
- ✗ Ein DELETE in der Primärtabelle ist immer möglich, ohne die Werte in der Detailtabelle zu beachten.
- ✓ Ein FOREIGN KEY wird so definiert: Er wird der Tabelle mit dem Fremdschlüssel und der betreffenden Spalte zugeordnet und verweist auf die Tabelle mit dem Primärschlüssel und der betreffenden Spalte.
- ✗ Ein FOREIGN KEY kann nicht unmittelbar bei der Definition der Fremdschlüssel-Spalte angegeben werden.
- ✗ Eine Fremdschlüssel-Beziehung kann nur zu jeweils einer Spalte der beiden Tabellen gehören.



Referentielle Integrität

- Erläutern Sie das Konzept der **Referentiellen Integrität** in relationalen Datenbanksystemen?

Referentielle Integrität



- Referentielle Integritätsbedingungen entstehen zwischen **Primär-** und **Fremdschlüssel**.

Beispiel

```
CREATE TABLE Professor(  
  PersNr    INTEGER PRIMARY KEY,  
  Name      VARCHAR(30) NOT NULL,  
  ...);
```

} Vartertabelle

```
CREATE TABLE Vorlesung(  
  VorlNr    INTEGER PRIMARY KEY,  
  Titel     VARCHAR(30),  
  SWS       INTEGER,  
  gelesenVon INTEGER,  
  CONSTRAINT FK_gelesenVon  
    FOREIGN KEY (gelesenVon)  
    REFERENCES Professor(PersNr)  
);
```

} Kindtabelle

- Zu jedem **Fremdschlüsselwert** der referenzierenden Kindtabelle (Vorlesung) muss ein entsprechender **Schlüsselwert** in der referenzierten Vartertabelle (Professor) existieren.

Referentielle Integrität

Beispiel

```
CREATE TABLE Abteilung (  
  AbtName VARCHAR(30),  
  CONSTRAINT PK_AbtName PRIMARY KEY (AbtName));
```

```
CREATE TABLE Mitarbeiter (  
  Name CHAR(30),  
  GebDatum DATE,  
  AbtName VARCHAR(30),  
  CONSTRAINT PK_Name_GebDatum PRIMARY KEY (Name, GebDatum),  
  CONSTRAINT FK_AbtName FOREIGN KEY (AbtName)  
    REFERENCES Abteilung (AbtName));
```

Abteilung

AbtName
IT
Einkauf
Marketing

Mitarbeiter

Name	GebDatum	AbtName
Koch	7.5.1966	IT
Neuper	1.2.1967	Marketing
Kogler	3.6.1968	Marketing

Welche der folgenden DML-Befehle sind erlaubt?

Beachten Sie: Jeder Befehl arbeitet ausschließlich mit den oben angegebenen Datenbeständen, d. h. die Befehle sehen NICHT eventuelle Änderungen vorausgegangener Operationen.

		JA	NEIN	?
a)	INSERT INTO Mitarbeiter VALUES ('Schnabl','3.7.1962','Org');			
b)	DELETE FROM Abteilung WHERE AbtName = 'IT';			
c)	DELETE FROM Mitarbeiter WHERE AbtName = 'IT';			
d)	UPDATE Abteilung SET AbtName = 'Org' WHERE AbtName = 'IT';			
e)	UPDATE Mitarbeiter SET AbtName = 'Org' WHERE AbtName = 'IT';			

Referentielle Integrität

Beispiel

```
CREATE TABLE Abteilung (  
  AbtName VARCHAR(30),  
  CONSTRAINT PK_AbtName PRIMARY KEY (AbtName));
```

```
CREATE TABLE Mitarbeiter (  
  Name CHAR(30),  
  GebDatum DATE,  
  AbtName VARCHAR(30),  
  CONSTRAINT PK_Name_GebDatum PRIMARY KEY (Name, GebDatum),  
  CONSTRAINT FK_AbtName FOREIGN KEY (AbtName)  
    REFERENCES Abteilung (AbtName));
```

Abteilung

AbtName
IT
Einkauf
Marketing

Mitarbeiter

Name	GebDatum	AbtName
Koch	7.5.1966	IT
Neuper	1.2.1967	Marketing
Kogler	3.6.1968	Marketing

Welche der folgenden DML-Befehle sind erlaubt?

Beachten Sie: Jeder Befehl arbeitet ausschließlich mit den oben angegebenen Datenbeständen, d. h. die Befehle sehen NICHT eventuelle Änderungen vorausgegangener Operationen.

		JA	NEIN	?
a)	INSERT INTO Mitarbeiter VALUES ('Schnabl','3.7.1962','Org');		✓	
b)	DELETE FROM Abteilung WHERE AbtName = 'IT';		✓	
c)	DELETE FROM Mitarbeiter WHERE AbtName = 'IT';	✓		
d)	UPDATE Abteilung SET AbtName = 'Org' WHERE AbtName = 'IT';		✓	
e)	UPDATE Mitarbeiter SET AbtName = 'Org' WHERE AbtName = 'IT';		✓	



Referentielle Aktionen

- Was sind **Referenzielle Aktionen**, wozu werden sie eingesetzt?

Referentielle Aktionen

- Klauseln **ON DELETE** und **ON UPDATE** geben an, welche referentiellen Aktionen bei einem **DELETE** bzw. **UPDATE** auf die referenzierte Vartertabelle ausgeführt werden sollen, um die referentielle Integrität zu gewährleisten

Beispiel

```
CREATE TABLE Professor(  
  PersNr      INTEGER PRIMARY KEY,  
  Name        VARCHAR(30) NOT NULL,  
  ...);  
  
CREATE TABLE Vorlesung(  
  VorlNr      INTEGER PRIMARY KEY,  
  Titel       VARCHAR(30),  
  SWS         INTEGER,  
  gelesenVon  INTEGER,  
  CONSTRAINT FK_gelesenVon  
    FOREIGN KEY (gelesenVon)  
    REFERENCES Professor(PersNr)  
    ON DELETE CASCADE/SET NULL/SET DEFAULT  
);
```

Referentielle Aktion - wird auf
Vorlesung-Tabelle (=Kindtabelle)
ausgeführt

Referentielle Aktionen

- Gegeben ist der **CREATE TABLE**-Befehl für die Relation **Vorgesetzte**.

Vorgesetzte

```
CREATE TABLE Vorgesetzte (  
  Angestellter VARCHAR(20) NOT NULL,  
  Vorgesetzter VARCHAR(20) NOT NULL,  
  CONSTRAINT PK_ANGESTELLTER PRIMARY KEY (ANGESTELLTER, VORGESETZTER),  
  CONSTRAINT FK_VORGESETZTER FOREIGN KEY (VORGESETZTER)  
    REFERENCES Vorgesetzte (ANGESTELLTER) ON DELETE CASCADE);
```

- Ein beispielhafter Datenbestand für die Relation **Vorgesetzte** ist in folgender Tabelle dargestellt:

Vorgesetzte

<u>Angestellter</u>	<u>Vorgesetzter</u>
Schulz	Schulz
Meier	Schulz
Müller	Meier
Schmidt	Schulz
Tulpe	Müller
Real	Tulpe
Taris	Müller

- Wie wirkt sich das Löschen des Datensatzes (Meier, Schulz) aus? Welche Datensätze bleiben übrig?

Referentielle Aktionen

- Gegeben ist der **CREATE TABLE**-Befehl für die Relation **Vorgesetzte**.

Vorgesetzte

```
CREATE TABLE Vorgesetzte (  
  Angestellter VARCHAR(20) NOT NULL,  
  Vorgesetzter VARCHAR(20) NOT NULL,  
  CONSTRAINT PK_ANGESTELLTER PRIMARY KEY (ANGESTELLTER),  
  CONSTRAINT FK_VORGESETZTER FOREIGN KEY (VORGESETZTER)  
    REFERENCES Vorgesetzte (ANGESTELLTER) ON DELETE CASCADE);
```

- Ein beispielhafter Datenbestand für die Relation **Vorgesetzte** ist in folgender Tabelle dargestellt:

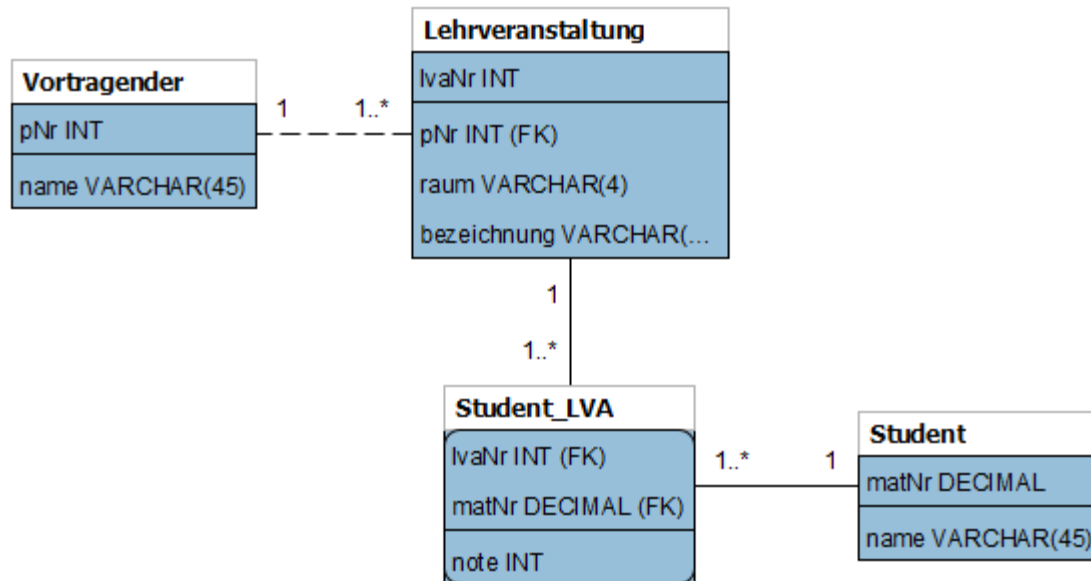
Vorgesetzte

<u>Angestellter</u>	<u>Vorgesetzter</u>
Schulz	Schulz
Meier	Schulz
Müller	Meier
Schmidt	Schulz
Tulpe	Müller
Real	Tulpe
Taris	Müller

- Wie wirkt sich das Löschen des Datensatzes (Meier, Schulz) aus? Welche Datensätze bleiben übrig?

DDL und SQL

- Im folgenden ER-Modell werden Lehrveranstaltungen modelliert. Ein Vortragender hält eine oder mehrere Lehrveranstaltungen, die von einem oder mehreren Studenten besucht werden. Studenten können auch mehrere Lehrveranstaltungen besuchen.
- Schreiben Sie ein entsprechendes SQL-Statement, um die Tabelle Lehrveranstaltung anzulegen. Definieren Sie falls nötig Primär- und Fremdschlüssel. Sorgen Sie außerdem dafür, dass Lehrveranstaltungen nicht stattfinden (gelöscht werden), falls der Vortragende kündigt (gelöscht wird).



DDL und SQL

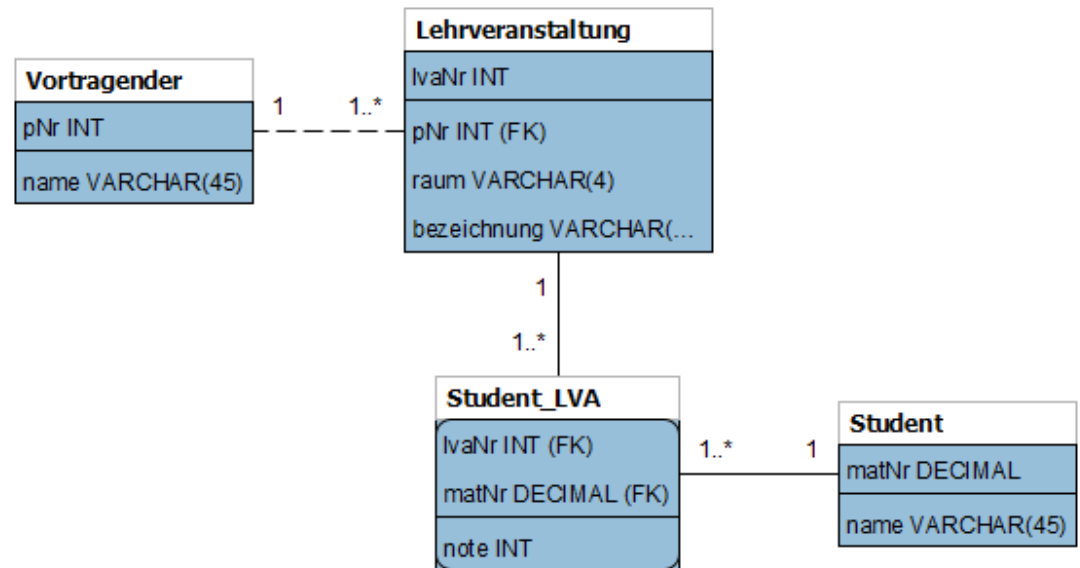


Tabelle Lehrveranstaltung

```
CREATE TABLE Lehrveranstaltung (  
  lvaNr INTEGER NOT NULL,  
  pNr INTEGER NOT NULL,  
  raum VARCHAR(4) NOT NULL,  
  bezeichnung VARCHAR(45) NOT NULL,  
  CONSTRAINT pk_Lehrveranstaltung PRIMARY KEY (lvaNr),  
  CONSTRAINT fk_Vortragender FOREIGN KEY (pNr)  
    REFERENCES Vortragender (pNr)  
    ON DELETE CASCADE  
);
```

Natürlicher Verbund vs. allgemeiner Verbund

■ Natürlicher Verbund $\bowtie$

- Implizite Gleichheitsbedingung auf gleichnamigen Attributen
- Die Gleichnamigen Attribute tauchen im Ergebnis nur einmal auf

■ Allgemeiner Verbund (Theta-Join) $\bowtie_{\theta}$

- Explizite (beliebige) (Join-) Verbundbedingung
- Im Falle von Inner- und Outer-Join werden alle Attribute der beiden Eingaberelationen in das Ergebnis projiziert.

Relationale Algebra

- Betrachten Sie das folgende Datenbankschema einer Prüfwerkstatt (Primär- und Fremdschlüsseln bzw. Primär-/Fremdschlüsseln):

Fahrzeug(nr, typ, besitzer, baujahr)

Mitarbeiter(mid, stufe, stunden, svnr)

Zertifikat(name, behörde, lvl)

Gutachten(nr: Fahrzeug.nr, name: Zertifikat.name,
mid: Mitarbeiter.mid, datum, ergebnis)

- Bei einem Gutachten beurteilt ein Mitarbeiter, ob ein bestimmtes Fahrzeug die Anforderungen eines bestimmten Zertifikats erfüllt.
- Formulieren Sie die folgenden Anfragen an die Datenbank mit Hilfe der Relationalen Algebra.

Relationale Algebra

```
Fahrzeug(nr, typ, besitzer, baujahr)
Mitarbeiter(mid, stufe, stunden, svnr)
Zertifikat(name, behörde, lvl)
Gutachten(nr: Fahrzeug.nr, name: Zertifikat.name,
           mid: Mitarbeiter.mid, datum, ergebnis)
```

- Es sollen alle Gutachten ausgegeben werden in denen ein/e MitarbeiterIn ein Zertifikat überprüft hat, für welches er/sie eigentlich nicht qualifiziert ist (d.h. das **lvl** des Zertifikates ist höher als die **stufe** des Mitarbeiters).

Geben Sie für jedes betroffene Gutachten zumindest die nötigen Informationen aus, um das Gutachten eindeutig zu identifizieren.

$$R := \pi_{nr, name, mid, datum}(\sigma_{stufe < lvl}(Gutachten \bowtie Zertifikat \bowtie Mitarbeiter))$$

Relationale Algebra

```
Fahrzeug(nr, typ, besitzer, baujahr)
Mitarbeiter(mid, stufe, stunden, svnr)
Zertifikat(name, behörde, lvl)
Gutachten(nr: Fahrzeug.nr, name: Zertifikat.name,
           mid: Mitarbeiter.mid, datum, ergebnis)
```

- Es sollen die Mitarbeiter-IDs (**mid**) aller MitarbeiterInnen ausgegeben werden, welche bereits Gutachten zu mindestens zwei verschiedenen Fahrzeugen erstellt haben.

Achten Sie darauf, dass es sich wirklich um mindestens zwei verschiedene Fahrzeuge handelt. Zwei Gutachten zum selben Fahrzeug gelten nicht.

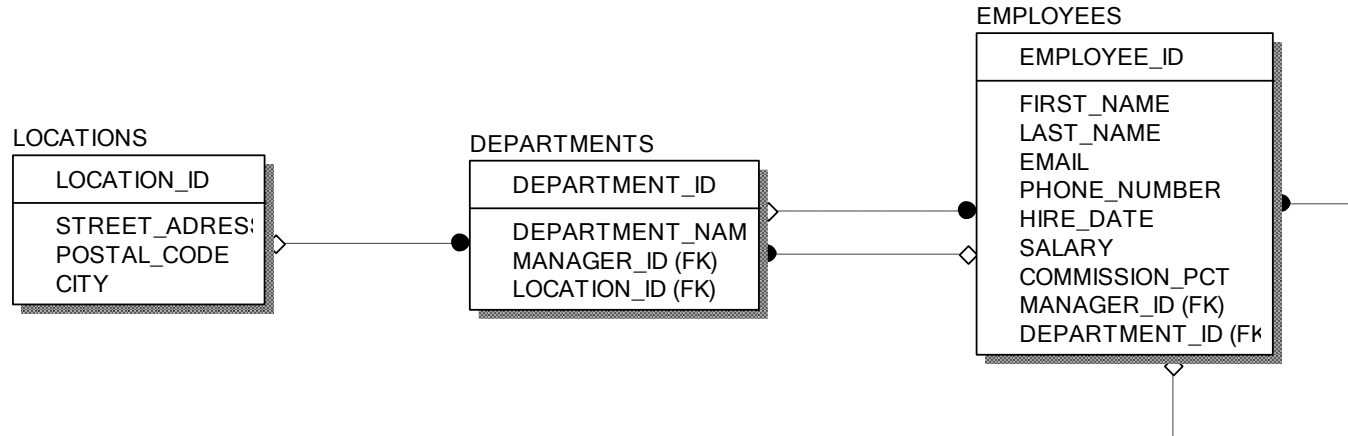
$$R := \pi_{G1.mid}(\sigma_{G1.mid=G2.mid \wedge G1.nr \neq G2.nr}(\beta_{G1}(Gutachten) \bowtie \beta_{G2}(Gutachten)))$$

„Beliebte“ Fehler in SQL-Anweisungen

- Verwechseln Sie nicht **ORDER BY** und **GROUP BY**, auch wenn es so ähnlich klingt.
- **ORDER BY** veranlasst lediglich das Sortieren der Ergebnistabelle einer **SELECT**-Anweisung nach den hinter **ORDER BY** stehenden Optionen, z. B.
`SELECT * FROM Student ORDER BY Name, Vorname`
- **GROUP BY** bezeichnet eine Gruppe, auf die eine oder mehrere Aggregations-Funktionen angewandt werden (**SUM**, **AVG**, **COUNT**, **MIN**, **MAX**). **GROUP BY** ist also immer nur in Verbindung mit mindestens einer Aggregationsfunktion sinnvoll. Ein Beispiel könnte die Berechnung der Gesamt-Durchschnittsnote jedes Studierenden (alle Noten eines Studierenden sind dann immer eine Gruppe) sein, z. B.
`SELECT Student.MatrNr, Student.Name, AVG(Noten.Note) AS
DurchschNote
FROM Student INNER JOIN NOTEN ON Student.MatrNr=Noten.MatrNr
GROUP BY Student.MatrNr, Student.Name;`
- Natürlich könnte das Ergebnis noch nach Namen sortiert werden. Sie brauchen dann nur die Zeile "**ORDER BY Student.MatrNr, Student.Name**" als letzte Zeile dazu schreiben.

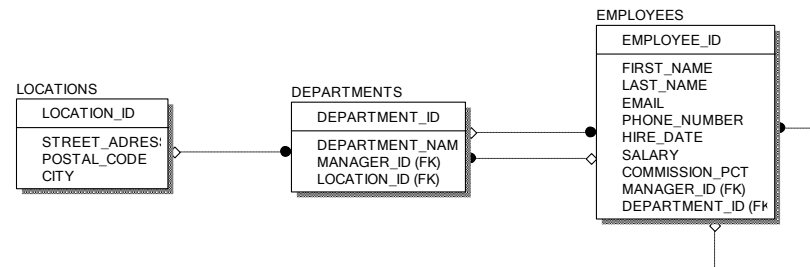
SQL-Abfragen

- Geben Sie die Namen und die Gehälter jener Mitarbeiter aus, die mehr verdienen als das durchschnittliche Gehalt in ihrer Abteilung?



SQL-Abfragen

- Geben Sie die Namen und die Gehälter jener Mitarbeiter aus, die mehr verdienen als das durchschnittliche Gehalt in ihrer Abteilung?

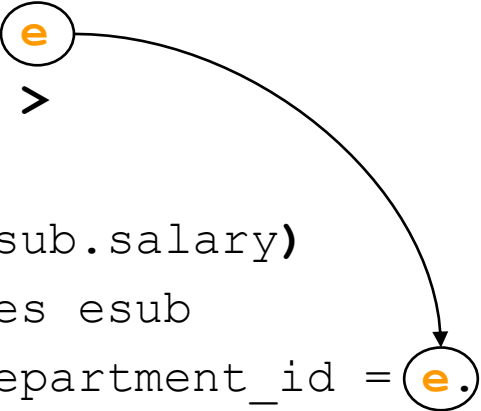


```
SELECT e.first_name, e.last_name, e.salary
FROM employees e
WHERE e.salary >
(
    SELECT AVG(esub.salary)
    FROM employees esub
    WHERE esub.department_id = e.department_id
);
```

Korrelierte Abfragen

- Was sind korrelierte Abfragen? Erklären Sie den Ablauf einer korrelierten Abfrage anhand eines Abfrage.

```
SELECT e.first_name, e.last_name, e.salary, e.department_id
FROM employees e
WHERE e.salary >
(
  SELECT AVG(esub.salary)
  FROM employees esub
  WHERE esub.department_id = e.department_id
);
```



Gruppierungen

- Erklären Sie die Funktionsweise der **HAVING**-Klausel in einer **GROUP BY**-Abfrage?
- In welchen Fällen verwenden Sie die **HAVING**-Klausel bzw. die **WHERE**-Klausel in einer **GROUP BY**-Abfrage?
Gibt es dabei Unterschiede?

Gruppierung

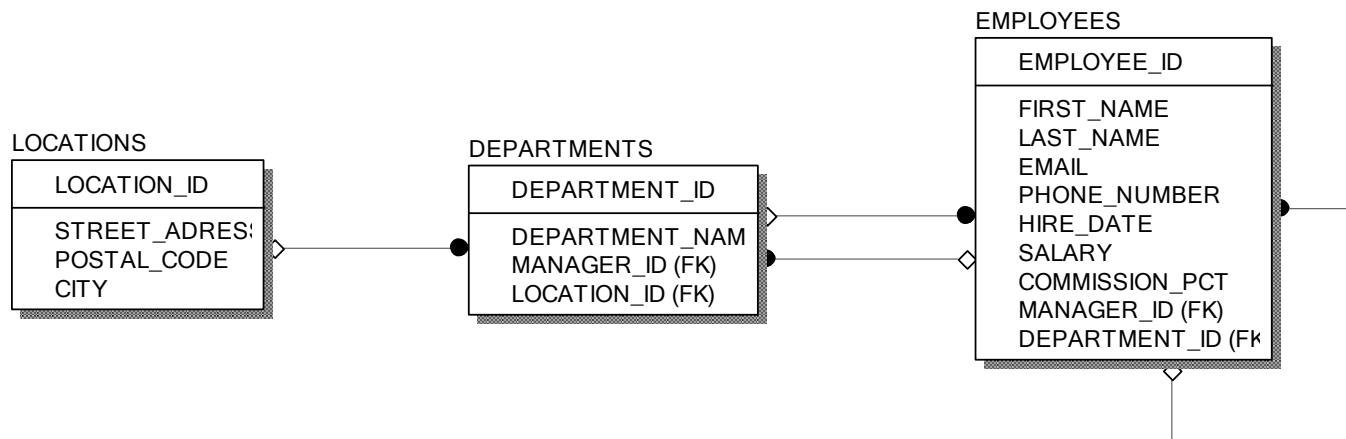
Notation:

```
SELECT ...  
FROM ...  
[ WHERE bedingungen ] —————> Auswahl Tabellenzeilen  
[ GROUP BY attributliste ] —————> Gruppierungen von Tabellenzeilen  
[ HAVING bedingungen ] ; —————> Auswahl Gruppierungen
```

- **GROUP BY**-Operator unterteilt die in der **FROM**-Klausel angegebene Tabelle in Gruppen, so dass alle Tupel innerhalb einer Gruppe den gleichen Wert für die **GROUP BY**-Attribute haben.
- Dabei werden alle Tupel, die die **WHERE**-Klausel nicht erfüllen, eliminiert.
- Danach werden innerhalb jeder solchen Gruppe die Aggregatfunktionen berechnet.
- Die **HAVING**-Klausel ermöglicht es, Bedingungen an die durch **GROUP BY** gebildeten Gruppen zu formulieren.
- In der **SELECT**-Klausel dürfen nur zwei Spaltenarten vorkommen
 - die, die mit einer Aggregatfunktion versehen sind und
 - die anderen müssen in der **GROUP BY**-Klausel enthalten sein.

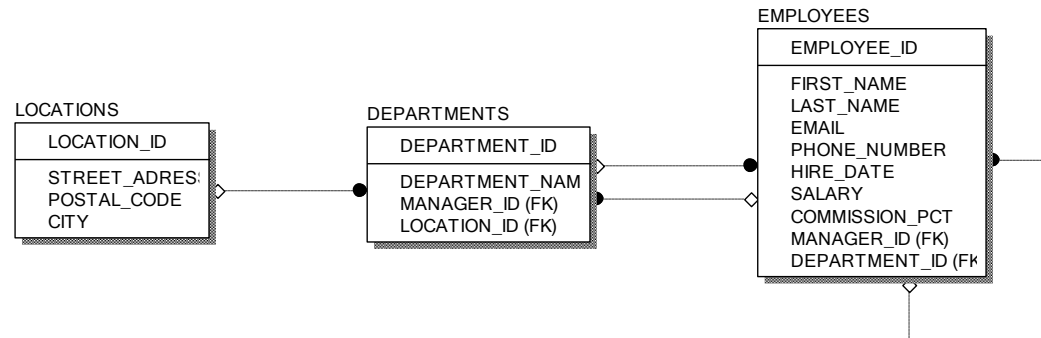
Aggregate und Gruppierung

- Geben Sie jene Abteilung aus (**DEPARTMENT_NAME**), deren Mitarbeiter durchschnittlich mehr als 10.000 verdienen. Geben Sie zusätzlich zum Abteilungsnamen (**DEPARTMENT_NAME**) auch die Anzahl der Mitarbeiter und den durchschnittlichen Verdienst dieser Mitarbeiter an.



Aggregate und Gruppierung

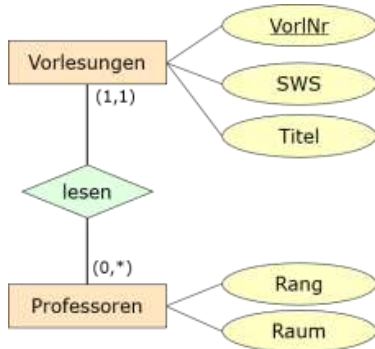
- Geben Sie jene Abteilung aus (**DEPARTMENT_NAME**), deren Mitarbeiter durchschnittlich mehr als 10.000 verdienen. Geben Sie zusätzlich zum Abteilungsnamen (**DEPARTMENT_NAME**) auch die Anzahl der Mitarbeiter und den durchschnittlichen Verdienst dieser Mitarbeiter an.



```
SELECT department_name, AVG(salary) , COUNT(*)  
FROM departments INNER JOIN employees USING(department_id)  
GROUP BY department_id, department_name  
HAVING AVG(salary) > 10000;
```

Aggregate und Gruppierung

- Universitätsdatenbank: Geben Sie zu jedem/r C4-ProfessorIn, der/die vor allem lange Vorlesungen (Durchschnitt größer gleich 3) hält, die Summe der von ihm/ihr gehaltenen Vorlesungsstunden an?



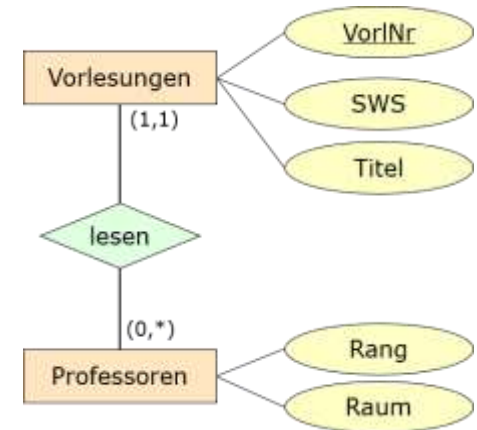
Professor

<u>PersNr</u>	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

Vorlesung

<u>VorlNr</u>	Titel	SWS	<u>GelesenVon</u>
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

Aggregate und Gruppierung



Beispiel:

Geben Sie zu jedem/r C4-ProfessorIn, der/die vor allem lange Vorlesungen (Durchschnitt größer gleich 3) hält, die Summe der von ihm/ihr gehaltenen Vorlesungsstunden an?

```
SELECT gelesenVon, Name, SUM(SWS)
FROM Vorlesung INNER JOIN Professor ON gelesenVon = PersNr
WHERE Rang = 'C4'
GROUP BY gelesenVon, Name
HAVING AVG(SWS) >= 3;
```



NULL-Werte

- Welche Bedeutungen hat ein NULL-Wert in einer relationalen Datenbank?

NULL-Werte

- **NULL** repräsentiert die Bedeutung
 - „Wert unbekannt“,
 - „Wert nicht anwendbar“ oder
 - „Wert existiert nicht“,
 - gehört zu keinem Wertebereich
- Kennzeichnung von Nullwerte in SQL durch **NULL**
- **NULL** kann in allen Spalten auftreten, außer in Schlüsselattributen und den mit **NOT NULL** gekennzeichneten

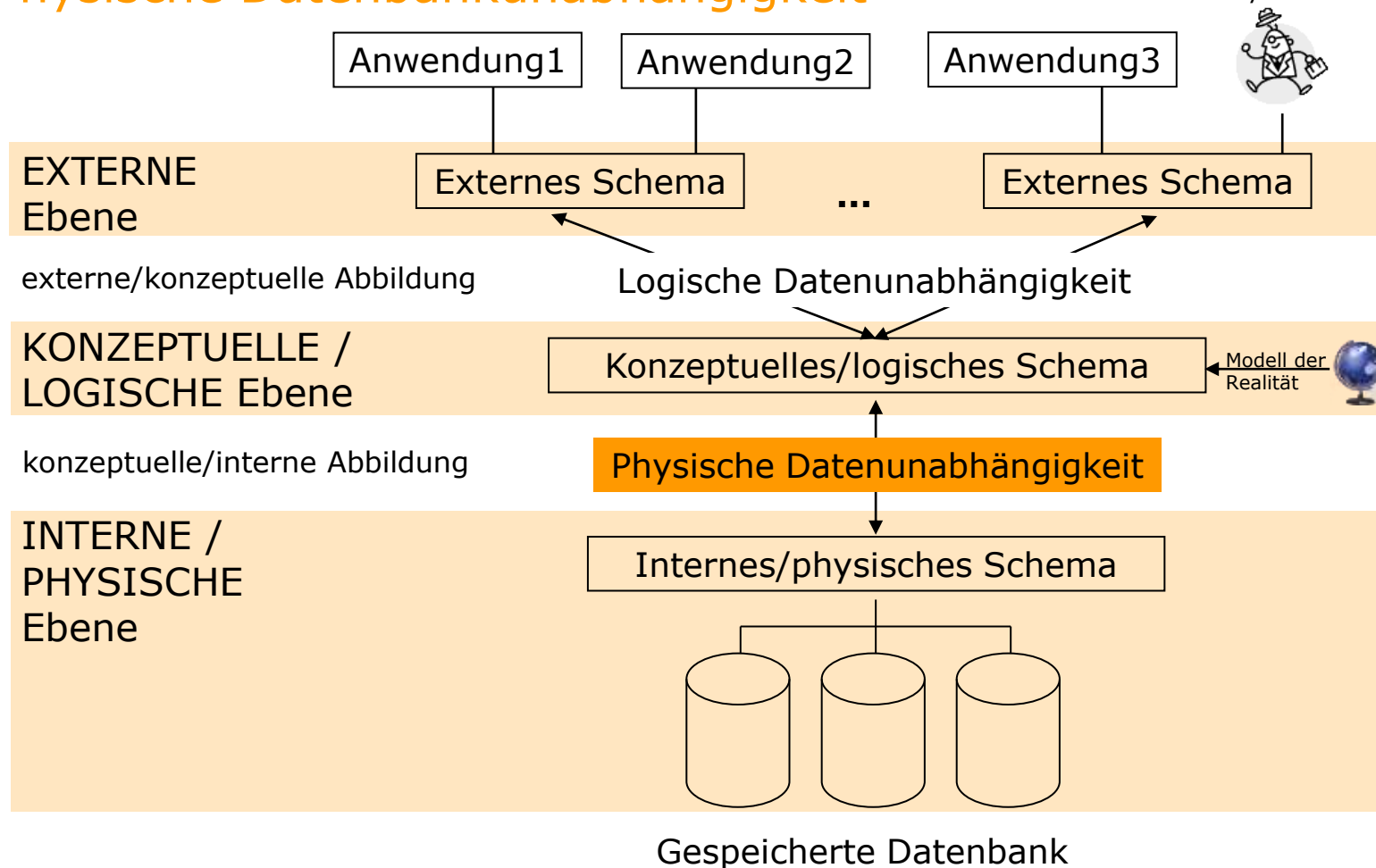
Beispiel

Name **VARCHAR(30) NOT NULL**

3-Ebenen-Architektur von Datenbanken

Physische Datenbankunabhängigkeit

ANSI/X3/SPARC: Normungsvorschlag
für Systemarchitektur von DBMS

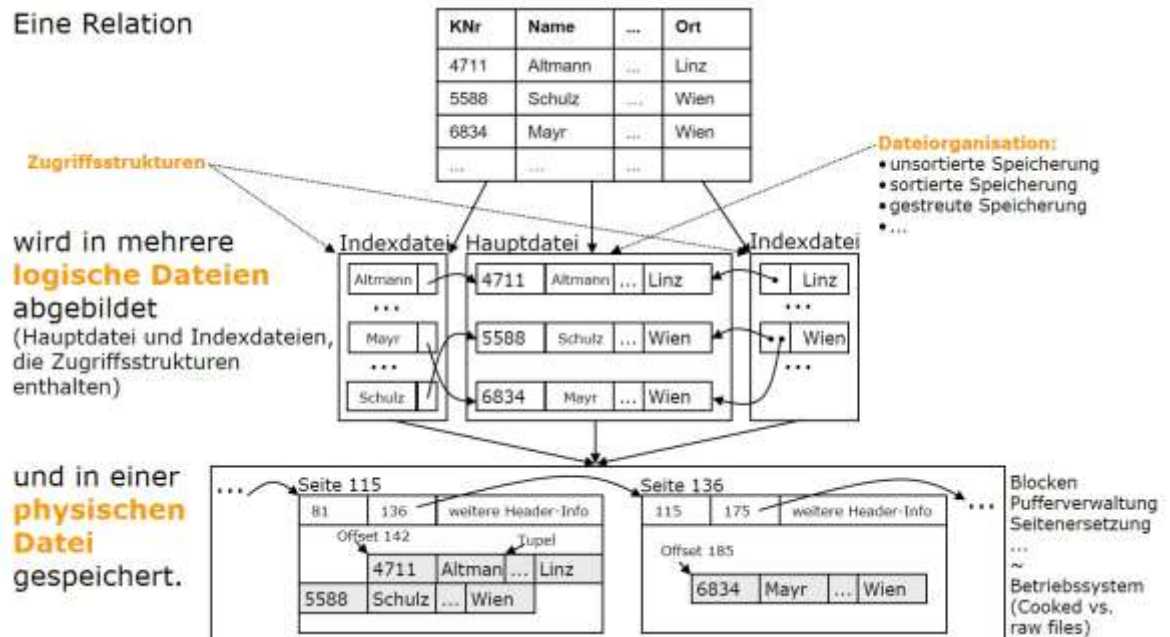


- Änderungen an der physischen Speicher- oder der Zugriffsstruktur (beispielsweise das Anlegen oder Entfernen einer Indexstruktur) hat keine Auswirkungen auf das logische/konzeptuelle Schema.

3-Ebenen-Architektur von Datenbanken

Physische Datenbankunabhängigkeit

- Was bedeutet der Begriff **Index** auf der internen/physischen Ebene der 3-Ebenen-Architektur?
- Ein Index ist eine zusätzlich abgelegte Datenstruktur, die die Werte einer oder mehrere Spalten/Attribute einer Tabelle/Relation indiziert um einen schnelleren Suchzugriff zu ermöglichen.



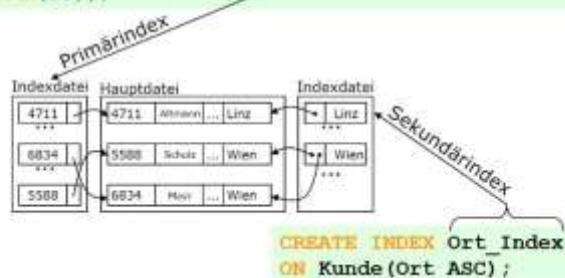
3-Ebenen-Architektur von Datenbanken

Physische Datenbankunabhängigkeit

- Worin liegt der Unterschied zwischen einem **Primär-** und **Sekundärindex**?
- **Primärindex**: wird normalerweise automatisch vom Datenbankmanagementsystem erstellt und über den Primärschlüsselattributen definiert. Ein Primärindex muss eindeutig sein (wird auch für **UNIQUE**-Spalten eingesetzt)
- **Sekundärindex**: bezeichnet man jeden weiteren Zugriffspfad auf eine Relation (bspw. für einen **FOREIGN KEY**-Spalte). Ein Sekundärindex muss nicht eindeutig sein.

Beispiel

```
CREATE TABLE Kunde  
( KNr NUMBER(4) PRIMARY KEY USING INDEX  
  (CREATE UNIQUE INDEX Kunde_KNr_Index ON Kunde(KNr)),  
  Name VARCHAR(20), ...,  
  Ort VARCHAR(20));
```



3-Ebenen-Architektur von Datenbanken

Physische Datenbankunabhängigkeit

- Geben Sie Beispiele an, wann ein Index verwendet werden sollte?
- **Kandidaten** für Indizierung
 - Primärschlüssel und `UNIQUE`-Spalten (automatisch!)
 - Fremdschlüssel-Spalten (meistens, vor allem bei Verbund!)
 - Spalten, die häufig in `WHERE`-Klauseln von signifikanten, selektiven Anweisungen verwendet werden
 - Spalten, die in der `ORDER BY`-Klausel oder von Funktionen wie `MAX` oder `MIN` verwendet werden
- **Selektivität**
 - = Prozentsatz der Werte einer Spalte mit gleichen Ausprägungen
 - *Faustregel*: Index nur bei Selektivität von bis zu ca. 10%, d.h. bei kleiner Treffermenge
- **Vorsicht bei**
 - Hoher `INSERT/UPDATE/DELETE`-Frequenz auf indizierte Attribute, da Index **reorganisiert** werden muss
 - Basistabellen mit wenigen Datensätzen (Index ab ~ 200 Zeilen)

Sichten

- Erklären Sie das Sichten-Konzept in relationalen Datenbanken und führen Sie Einsatzmöglichkeiten an?
- Welche Arten von Sichten gibt es? Beschreiben Sie diese?
- Welche Voraussetzungen müssen virtuelle Sichten besitzen, damit diese änderbar sind?

Was ist eine Sicht?

⇒ Stellt **Teilmengen von Daten** aus einer oder mehreren Tabellen (Basistabellen, Sichten) logisch dar.

└─ = mit **CREATE TABLE** definierte Tabellen

■ Sicht = **Virtuelle Tabelle**

■ Was bedeutet "virtuell"?

- Tabelle kann wie eine Basistabelle verwendet werden
 - `SELECT ... FROM VirtuelleTabelle WHERE ...`
 - jedoch mit Einschränkungen: `INSERT`, `DELETE`, `UPDATE`
- Tabelle ist nicht permanent gespeichert
- Tabelleninhalt wird bei jeder Verwendung aus anderen Tabellen neu berechnet
- ➔ "**dynamisches Fenster**" auf zugrunde liegende Basistabellen und Sichten

Einsatzgebiete

- Komplexe Anfragen erleichtern
... durch einfache Abfrage der Sicht (~ "Makro")
- Strukturierung der Datenbank
... zugeschnitten auf Benutzergruppen
- Logische Datenunabhängigkeit gewährleisten
... Sichten stabil bei Änderungen der Datenbankstruktur
und umgekehrt bei Änderungen der Anwendungen
- Datenzugriff einschränken
... Sicherheits- und Autorisierungsmechanismus

Sichtarten und Einschränkungen bzgl. Änderbarkeit

■ Vier verschiedene Sichtarten

- Projektion $\Rightarrow$ **Projektionssicht**
- Selektion $\Rightarrow$ **Selektionssicht**
- Verbund $\Rightarrow$ **Verbundssicht**
- Aggregation $\Rightarrow$ **Aggregationssicht**

■ Faustregel

- Auf einer Sicht sind nur solche DML-Operationen zulässig, zu denen eine **eindeutige Transformation auf die Basisrelationen** existiert.